



Homework 7

Please complete the following problems. You must submit the following for each problem: a) the answer you are asked to return and b) Python code that can be run to verify your work.

1. Creating Phylogenetic Trees on the Command Line

One of the most popular phylogenetic inference algorithms that is not in MEGA is MrBayes. “MrBayes is a program for Bayesian inference and model choice across a wide range of phylogenetic and evolutionary models. MrBayes uses Markov chain Monte Carlo methods to estimate the posterior distribution of model parameters.”

There are many programs for building phylogenetic trees and many file formats used by these programs. MrBayes uses the NEXUS (.nex) file format. A NEXUS file is made of a fixed header `#NEXUS` followed by multiple blocks. Each block starts with `BEGIN block_name;` and ends with `END;`.

There are a few pre-defined block names for common types of data. The DATA block type contains a data matrix. Remember the data input required for creating a phylogenetic tree is a multiple sequence alignment (MSA). The DATA block matrix in a NEXUS file can contain an MSA.

PROBLEM

The MrBayes program can be downloaded [here](#) and run via the command line. For help running bioinformatics programs on the command line, check out this [resource](#).

Given: A NEXUS file containing an MSA of 20 orthologous protein sequences.

Return: The cladogram generated by the MrBayes program after 300,000 generations. Use the following settings when running the analysis:

```
MrBayes > lset nucmodel=protein
MrBayes > prset aamodelpr=mixed
MrBayes > mcmc nchains=1 ngen=300000
```

These settings represent a basic, fast method for creating protein trees.

Hint: Consult the MrBayes manual that is downloaded with the program for help running the analysis and details on the desired settings. Follow the typical Bayesian phylogenetic analysis protocol. The input file is loaded with the `execute` command. The evolutionary model is set with the `lset` and `prset` commands. The phylogenetic analysis is run with the `mcmc` command. The output trees (a cladogram and a phylogram) are generated with the `sumt` command.

2. FASTA Format Introduction

Modern sequencing instruments generally produce an unbelievable number of reads, which are generally short sequences of <200 bases. All sequencing machines are prone to some degree of error, so it is important to quantify the certainty with which each read is identified, or "called." This is often referred to as the quality of the base call.

This quality is customarily measured on the Phred quality scale. Lower scores indicate less confidence in the accuracy of the base call. The Phred quality score Q is calculated using the following equation:

$$Q = -10 \log_{10} P$$

where P is the probability that the corresponding base call is incorrect. Thus, for $Q = 10$, the probability that the base is called incorrectly is 1 in 10, and the accuracy is 90%. For $Q = 20$, the probability of error is 1 in 100 and the accuracy is 99%, and so on. See Figure 1 for an illustration.

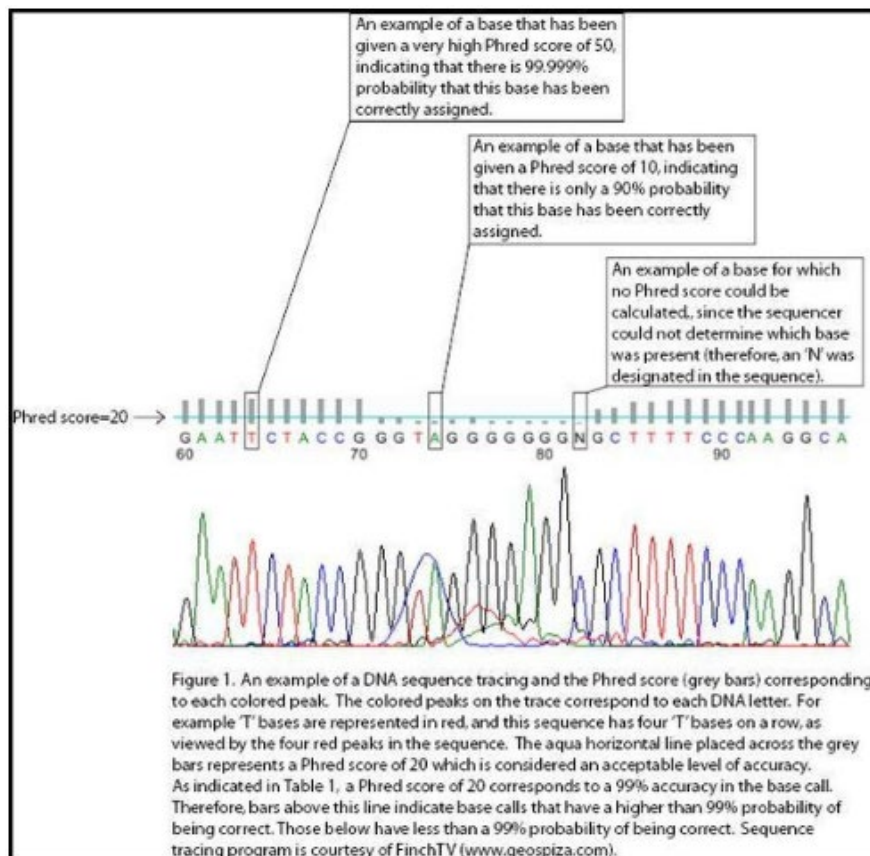


Figure 1: Phred quality scores.

The standard format for storing the output of high throughput sequencing instruments is the FASTQ format. FASTQ is an extension of FASTA format; it provides both the sequence and the per-base quality scores for each read. Nucleotides are represented by a single symbol, while Phred scores are usually 2-digit numbers. To be able to conveniently list the two together in FASTQ format, an offset of 33 is added to the score, which is then represented by the corresponding symbol from the ASCII table. Here are two examples:



- The quality of a base with a Phred score of 10 (90% accuracy) is denoted by "+" (ASCII symbol #43)
- The quality of a base with a Phred score of 40 (99.99% accuracy) is denoted by "I" (ASCII symbol #73).

The offset of 33 was chosen because the symbols 0-33 in the ASCII table are reserved for a non-printable characters, such as backspace or vertical tab.

Each record in a FASTQ file typically consists of four lines:

- A line starting with @ that contains the sequence identifier
- The actual sequence
- A line starting with + containing an optional sequence identifier or comment
- A line with quality scores encoded as ASCII symbols

Notice that the 2nd and 4th line should always have the same length.

The following is an example FASTQ record:

```
@HWI-ST999:102:D1N6AACXX:1:1101:1235:1936 1:N:0:
ATGTCTCCTGGACCCCTCTGTGCCAAGCTCCTCATGCATCCTCCTCAGCAACTTGTCTGTAGCTGAGGCTCACTGACTACCAGCTGCAG
+
1:DAADDDF\<B\<AGF=FGIEHCCD9DG=1E9?D>CF@HHG??B\<GEBGHCG;;CDB8==C@@>>GII@@5?A?@B>CEDCFCC:;?CCCA
C
```

Generally, a FASTQ file has the suffix .fq or .fastq.

PROBLEM

Sometimes it's necessary to convert data from FASTQ format to FASTA format. For example, you may want to perform a BLAST search using reads in FASTQ format obtained from your own sequencing instrument.

Links:

- A FASTQ to FASTA converter can be accessed from the Sequence conversion [website](#)
- A free GUI converter developed by BlastStation is available [here](#) for download or as an add-on to the Google Chrome browser.
- There is a FASTQ to FASTA converter in Galaxy, an open web-based platform for biomedical research. It offers many tools for data analysis and allows one to build a multi-step data processing workflow. Galaxy supports data upload from the user's computer and from online resources. Galaxy is available as a free [public web server](#) and as open-source software that can be downloaded and installed for the local usage. Several organizations with their own Galaxy installation have also opted to make those servers available to others. List of such Galaxy servers can be found [here](#). To use the public Galaxy servers, you should register for a Galaxy account and upload the input file prior to doing the conversion. Note that it can take a lot of time to complete a task depending on how busy the servers are with other people's work. I am well-aware that the Biostar Handbook calls Galaxy a “deeply misguided effort” and a “dead-end” solution. You are encouraged to download and install command line tools and work toward being “computationally proficient” whenever possible. Nonetheless, Galaxy can be a useful shortcut at times.



Homework

Bioinformatics Engineering

ENBC455

Given: A FASTQ file.

Return: Corresponding FASTA records.

Note: if you do not use Python to solve this problem, then please submit a screenshot of the results from your chosen conversion method.

Sample Dataset

```
@SEQ_ID
GATTGGGGTTCAAAGCAGTATCGATCAAATAGTAAATCCATTTGTTCAACTCACAGTTT
+
!*(((***+))%%%+)(%%%).1***-+*****)**55CCF>>>>>CCCCCCC65
```

Sample Output

```
>SEQ_ID
GATTGGGGTTCAAAGCAGTATCGATCAAATAGTAAATCCATTTGTTCAACTCACAGTTT
```

Hint: Python can do the conversion quite easily with `Bio.SeqIO.convert()`. The `Bio.SeqIO.convert()` function will take handles or filenames. Watch out though – if the output file already exists, it will overwrite it! To find out more, see the built in help:

```
from Bio import SeqIO
help(SeqIO.convert)
```

3. Read Quality Distribution

One of the first steps in the analysis of sequencing data is quality control. Unusual characteristics of the data may indicate problems in an earlier step, such as sample preparation, that must be corrected to obtain valid data.

The simplest metric to consider is the Phred quality score, introduced in the “FASTQ format introduction” problem. Since the FASTQ files containing Phred scores are flat and quite large (up to tens of gigabytes), special tools have been created for working with them.

Since a sequencing run can produce thousands or millions of reads, we are less interested in the quality scores for any particular read and instead want to examine the quality characteristics of the run as a whole. One common metric at this level is the per-read quality score distribution. The average quality of each read is calculated, and the distribution of these average scores can be plotted and analyzed.

In Figure 1, one can observe that most reads in a good sequencing run have an average quality near 32. Figure 2 is an example of bad dataset.

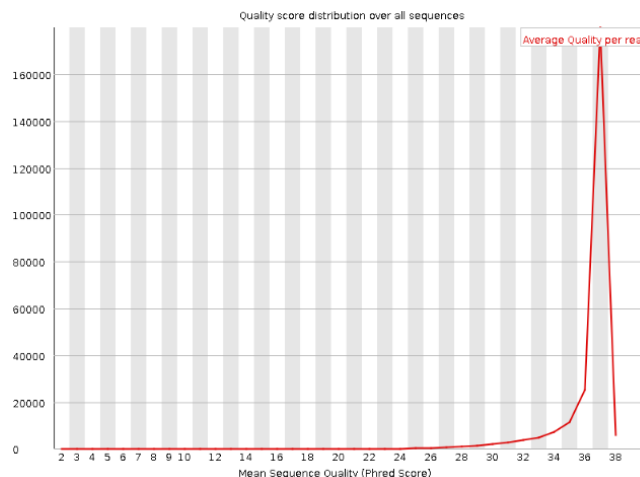


Figure 1: Per sequence quality scores - good data.

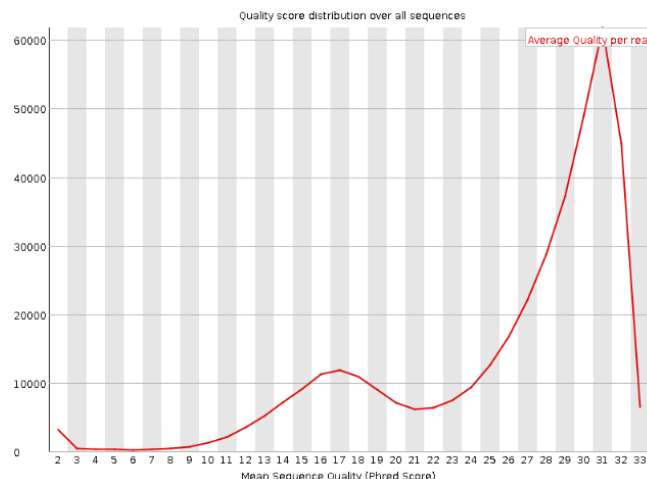


Figure 2: Per sequence quality scores - bad data.



One program for quality control of bulk sequence read data is FastQC, developed by Babraham Bioinformatics. It provides graphs and tables for quick quality assessment.

PROBLEM

A version of FastQC can be downloaded [here](#) and run locally on any operating system with a suitable Java Runtime Environment (JRE) installed. FastQC is also in Galaxy.

Given: A file containing a quality threshold, along with FASTQ entries for multiple reads.

Return: The number of reads whose average quality is below the threshold.

Note: if you do not use Python to solve this problem, then please submit a screenshot of the FASTQC results.

Sample Dataset

```
28
@Rosalind_0041
GGCCGGTCTATTACGTTCTCACCCGACGTGACGTACGGTCC
+
6.3536354;.151<211/0?::6/-2051)-*"40/.,+%)
@Rosalind_0041
TCGTATGCGTAGCACTTGGTACAGGAAGTGAACATCCAGGAT
+
AH@FGGGJ<GB<<9:GD=D@GG9=?A@DC=;;?:>839/4856
@Rosalind_0041
ATTCCGTAATTGGCGTGAATCTGTTCTGACTGATAGAGACAA
+
@DJEJEA?JHJ@8?F?IA3=;8@C95=;=?;>D/:;74792.
```

Sample Output

```
1
```

Hint: When used to read FASTQ data, BioPython's function `SeqIO.parse` returns a `SeqRecord` object containing the Phred quality scores corresponding to each base of the sequence. The scores are found in the `.letter_annotations` attribute, which is a Python dictionary having, in this case, a single key, 'phred_quality.'

```
>>> print(record.letter_annotations.keys())
['phred_quality']
>>> print(record.letter_annotations["phred_quality"])
[40, 39, 38, 37, 36, 35, 34, 13, 12, 11, 10, 9, 8]
```



4. Read Filtration by Quality

Filtering out poor-quality reads from a sequencing run can have multiple benefits. Not only is it likely to improve results, but it will generally reduce the time and memory overhead required for assembly or analysis.

PROBLEM

Poor-quality reads can be filtered out using the **FASTQ Quality Filter** tool from the FASTX toolkit. FASTX is a collection of command line tools for short reads FASTQ files preprocessing, developed by the Hannon Lab at Cold Spring Harbor. It allows one to prepare NGS data for further analysis by filtering out low-quality reads, trimming sequences based on quality, and performing many other tasks.

A command-line version of FASTX can be downloaded for Linux or MacOS from its [website](#). An online interface for the tool (called **Filter by quality**) is also available [here](#) within the Galaxy web platform.

Given: A file containing the quality threshold value q , the percentage of bases p , and set of FASTQ entries.

Return: Number of reads in filtered FASTQ entries.

Note: if you do not use Python to solve this problem, then please submit a screenshot of the results from your chosen filtering method.

Sample Dataset

```
20 90
@Rosalind_0049_1
GCAGAGACCAGTAGATGTGTTTGC GGACGGTCGGGCTCCATGTGACACAG
+
FD@@;C<AI?4BA:=>C<G=:AE=><A??>764A8B797@A:58:527+,
@Rosalind_0049_2
AATGGGGGGGGGAGACAAAATACGGCTAAGGCAGGGGTCCTTGATGTCAT
+
1<<65:793967<4:92568-34:.>1;2752)24')*15;1,.3*3+*!
@Rosalind_0049_3
ACCCCATACGGCGAGCGTCAGCATCTGATATCCTCTTTCAATCCTAGCTA
+
B:EI>JDB5=>DA?E6B@@CA?C;=;@@C:6D:3=@49;@87;;;?8+
```

Sample Output

```
2
```